```
docker run --log-driver=awslogs \
           --log-opt awslogs-region=eu-west-1 \
           --log-opt awslogs-group=docker-test \
           --log-opt awslogs-create-group=true \
       ...
```

If you want to learn more about how to use logs with containers, I recommend reading *Logging in Action* by Phil Wilkins (Manning, 2022; https://www.manning.com/books/logging-in-action, which covers this topic in great detail.

For metrics, there are some options available for containers:

- A rather popular and widely used solution to expose metrics is cAdvisor (http://mng.bz/9D4j), which is used together with Prometheus.
- Telegraf (http://mng.bz/WzRg) supports a Docker plug-in (https://github.com/ influxdata/telegraf/tree/master/plugins/inputs/docker) that generates rich metrics, including container stats, memory, CPU usage, and network-related metrics.
- Docker provides a native (https://docs.docker.com/config/daemon/prometheus/) Prometheus integration, which is limited to the runtime (that is, not covering the app, at time of writing).

Specifically, if you are using the Docker daemon (https://docs.docker.com/config/daemon/), you have the option to get (low-level) access to metrics via its socket, as shown in the following code. The output of the `curl` command is edited down to show some example stats; it's in JSON format, so you can query it directly, by using, for example, `jq`:

Launches a long-running container—in this case, the NGINX server. Note the container ID 82cea007263254659534e47cfb13b951f3426f6d91a16246143480f18a217718 here, which we need in the next step to query the metrics.

```
$ sudo docker run -d --rm  nginx
82cea007263254659534e47cfb13b951f3426f6d91a16246143480f18a217718

$ sudo curl --unix-socket /var/run/docker.sock \
    "http://localhost/v1.40/containers/82cea0072632/stats?stream=false"

{
 ... "system_cpu_usage":18084207160000000,"online_cpus":2,
 ... "rss":1892352,"rss_huge":0,"total_active_anon":1892352,
 ... "total_active_file":0,"total_cache":0,"total_dirty":0,
}
```

Use curl to query the UNIX socket /var/run/docker.sock. The Docker daemon facilitates providing services such as starting or stopping containers. We need to provide the container ID we want to have the metrics for, or at least a part that uniquely addresses the container (82cea0072632, in our case).

Table 3.2 shows some example metrics (from cAdvisor) that you can expect to find, typically from containers and their runtimes.